

Relational Database Design and Anomalies

1. Semantics of Relational Attributes

Guideline 1: Each tuple in a relation should represent one entity or relationship instance.
- Attributes of different entities (e.g., EMPLOYEES, DEPARTMENTS, PROJECTS) should not be mixed in the same relation. - Only foreign keys should be used to refer to other entities. - Entity and relationship attributes should be kept apart as much as possible.

Bottom Line: Design a schema that can be explained easily relation by relation. The semantics of attributes should be easy to interpret.

2. Redundant Information and Anomalies

Redundant information wastes storage and can cause anomalies:

- **Insertion Anomaly:** Cannot add data without unnecessary dummy or NULL values.
- **Update Anomaly:** Inconsistent data when updating duplicate information.
- **Deletion Anomaly:** Unintended loss of data when deleting other information.

3. Insertion Anomaly

Example Table (Students and Courses):

StudentID	StudentName	CourseID	CourseName
1	Alice	C101	Math
2	Bob	C101	Math
3	Charlie	C102	Physics

Scenario: We want to add a new course 'Biology (C103)' but no student is enrolled yet.

Problem: Cannot insert course without student info; would require NULL or dummy values.

Solution: Normalize into separate tables:

- Students(StudentID, StudentName)
- Courses(CourseID, CourseName)
- Enrollments(StudentID, CourseID)

4. Update Anomaly

Example Table:

StudentID	StudentName	CourseID	CourseName
1	Alice	C101	Math
1	Alice	C102	Physics
2	Bob	C101	Math

Scenario: Alice changes her name to 'Alicia'.

Problem: Name appears in multiple rows. Missing a row update causes inconsistency:

1	Alicia	C101	Math
1	Alice	C102	Physics

Solution: Store student info in a separate Students table to avoid redundancy.

5. Deletion Anomaly

Example Table:

StudentID	StudentName	CourseID	CourseName
1	Alice	C101	Math
2	Bob	C101	Math
3	Charlie	C102	Physics
4	David	C103	Chemistry

Scenario: David drops his only course (C103 - Chemistry).

Problem: Course info is also deleted because it is stored in the same table.

Solution: Separate courses into their own table.

6. Null Values in Tuples

Example Table (Employee Projects):

EmpID	EmpName	ProjID	ProjName	Hours
E1	Alice	P1	Billing	20
E2	Bob	P1	Billing	25
E3	Charlie	NULL	NULL	NULL

Problem: Charlie has no project assigned, so NULL values are used. Too many NULLs indicate poor design.

Solution: Separate employees and projects into different tables.

7. Spurious Tuples

Spurious tuples occur when a JOIN produces extra, incorrect rows.

Example:

Students Table:	StudentID	StudentName
	1	Alice
	2	Bob

Courses Table:	CourseID	CourseName
	C101	Math
	C102	Physics

Enrollments Table:	StudentID	CourseID
	1	C101
	2	C102

Incorrect JOIN without proper foreign keys might produce:

StudentID	StudentName	CourseID
1	Alice	C101
1	Alice	C102
2	Bob	C101
2	Bob	C102

Solution: Ensure **lossless join decomposition**. Only valid combinations appear.

8. Summary Table

Anomaly	Meaning	Cause	Solution
Insertion	Cannot add new info without dummy/NULL	Multiple entities in one table	Split tables
Update	Data becomes inconsistent	Repeated info	Normalize tables
Deletion	Losing important info	Multiple entities in one table	Split tables
Null Values	Missing/unknown data	Poor design	Separate optional attributes
Spurious Tuples	Wrong data after JOIN	Bad decomposition	Ensure lossless join

1. Definition of Functional Dependencies (FDs)

A Functional Dependency (FD) is a constraint between two sets of attributes in a relation:

$$X \rightarrow Y$$

This means: if two tuples have the same value for X, they must have the same value for Y.

Example: $SSN \rightarrow EName$ - Knowing SSN uniquely identifies the employee name.

2. Example Database (Employee-Project)

SSN	EName	PNumber	PName	PLocation	Hours
101	Alice	P1	Billing	NYC	20
102	Bob	P1	Billing	NYC	25
103	Charlie	P2	Research	Boston	15
101	Alice	P2	Research	Boston	10

3. Functional Dependencies in This Database

- **Single Attribute FDs:**

- $SSN \rightarrow EName$
- $PNumber \rightarrow \{PName, PLocation\}$

- **Composite Attribute FDs:**

- $\{SSN, PNumber\} \rightarrow Hours$
-

4. Explanation of Attributes and FDs

- **Attribute:** A property of an entity (column in table)
- **Functionally Determines:** Knowing X is enough to uniquely identify Y

Examples:

- $SSN \rightarrow EName$: knowing $SSN=101$, $EName=Alice$
 - $\{SSN, PNumber\} \rightarrow Hours$: knowing $SSN=101$ and $PNumber=P2$, $Hours=10$
-

5. Determining FDs from Instances

- Look at table data - Check which columns uniquely identify others - Example: SSN alone cannot determine Hours; SSN, PNumber can.

6. Summary (Lattice Study Sheet)

- Single Attribute FDs:
 - $SSN \rightarrow EName$
 - $PNumber \rightarrow \{PName, PLocation\}$
- Composite Attribute FDs:
 - $\{SSN, PNumber\} \rightarrow Hours$
- Why FDs matter:
 - Ensure data consistency
 - Help database normalization
 - Prevent redundancy and anomalies

1 Normalization of Data

Normalization is the process of analyzing relation schemas based on their functional dependencies (FDs) and primary keys to achieve two main objectives:

1. Minimize redundancy
2. Minimize insertion, deletion, and update anomalies

It can be considered as a “filtering” or “purification” process to improve the quality of database design. If a relation schema fails a normal form test, it is decomposed into smaller schemas that satisfy the test.

1.1 Benefits of Normalization

Normalization provides:

- A formal framework for analyzing relation schemas using keys and functional dependencies.
- A series of normal form tests to ensure the database can be normalized to the desired level.

1.2 Important Properties of Normalized Schemas

After decomposition, two critical properties should be considered:

- **Nonadditive Join (Lossless Join):** Guarantees that combining decomposed relations does not produce spurious tuples. This property must always be ensured.
- **Dependency Preservation:** Ensures each functional dependency is represented in some decomposed relation. This is desirable but can sometimes be sacrificed.

1.3 Practical Use of Normal Forms

- Most databases are derived from existing designs, including legacy systems.
- Normalization ensures high-quality and sustainable designs.
- In practice, normalization is usually done up to 3NF, BCNF, or occasionally 4NF.
- Higher normal forms (4NF, 5NF) are rare, complex, and harder to understand.
- **Denormalization:** Intentionally storing a join of higher-normal-form relations as a single relation (lower normal form) to improve performance.

1.4 Keys and Attributes

- **Superkey:** A set of attributes that uniquely identifies tuples in a relation.
- **Key:** A minimal superkey; removing any attribute breaks uniqueness.
- **Candidate key:** Any key of a relation.
- **Primary key:** Arbitrarily chosen candidate key.
- **Secondary keys:** Remaining candidate keys.
- **Prime attribute:** Member of a candidate key.
- **Nonprime attribute:** Not part of any candidate key.

1.5 First Three Normal Forms

- **1NF:** Atomic attribute values (no repeating groups)
- **2NF:** No partial dependency of nonprime attributes on a candidate key
- **3NF:** No transitive dependency of nonprime attributes on a candidate key

Note: 3NF inherently satisfies 1NF and 2NF. The sequence $(1NF \rightarrow 2NF \rightarrow 3NF)$ is followed for systematic normalization.

2 Conversion to First Normal Form (1NF)

First Normal Form (1NF) requires that all attributes of a relation be **atomic**, disallowing:

- Composite attributes
- Multivalued attributes
- Nested relations

Most RDBMSs only allow relations in 1NF to be defined. Below we demonstrate examples of converting unnormalized relations to 1NF.

2.1 Example 1: Students and Courses

Unnormalized Relation (UNF):

StudentID	Name	Courses
101	Alice	Math, Physics
102	Bob	Chemistry
103	Charlie	Physics, Math, Chemistry

Conversion to 1NF:

- Identify the multivalued attribute: **Courses**.
- Split each course into separate rows so that each cell contains a single value.

1NF Table:

StudentID	Name	Course
101	Alice	Math
101	Alice	Physics
102	Bob	Chemistry
103	Charlie	Physics
103	Charlie	Math
103	Charlie	Chemistry

2.2 Example 2: Employees and Projects

Unnormalized Relation (UNF):

EmpID	EmpName	Projects Assigned
201	John	P1, P2
202	Mary	P2
203	Steve	P1, P3, P4

Conversion to 1NF:

- Split the multivalued attribute **Projects Assigned** into separate rows.

1NF Table:

EmpID	EmpName	Project
201	John	P1
201	John	P2
202	Mary	P2
203	Steve	P1
203	Steve	P3
203	Steve	P4

2.3 Example 3: Customers and Phone Numbers

Unnormalized Relation (UNF):

CustomerID	CustomerName	PhoneNumbers
301	Alice Smith	12345, 67890
302	Bob Brown	54321
303	Carol White	98765, 43210, 11111

Conversion to 1NF:

- Split the PhoneNumbers attribute so that each phone number is atomic.

1NF Table:

CustomerID	CustomerName	PhoneNumber
301	Alice Smith	12345
301	Alice Smith	67890
302	Bob Brown	54321
303	Carol White	98765
303	Carol White	43210
303	Carol White	11111

2.4 Example 4: Composite Attributes and Nested Relations (Split Email and Phone)

Unnormalized Relation (UNF) with Composite and Nested Attributes:

Consider a table storing student information, including their address as a composite attribute and multiple contacts as a nested relation.

StudentID	Name	Address	Contacts
101	Alice	(Street: 12A, City: NY)	(Email: alice@mail.com, Phone: 12345)
102	Bob	(Street: 45B, City: LA)	(Email: bob@mail.com)
103	Charlie	(Street: 78C, City: SF)	(Email: charlie@mail.com, Phone: 67890, Phone: 54321)

Problems:

- **Composite attribute:** Address contains multiple components (Street, City).
- **Nested/multivalued attribute:** Contacts contains multiple values (Email, Phone), and some students have multiple phone numbers.

Step 1: Flatten Composite Attributes

Split the Address attribute into atomic components:

StudentID	Name	Street	City	Contacts
101	Alice	12A	NY	(Email: alice@mail.com, Phone: 12345)
102	Bob	45B	LA	(Email: bob@mail.com)
103	Charlie	78C	SF	(Email: charlie@mail.com, Phone: 67890, Phone: 54321)

Step 2: Flatten Nested/Multivalued Attributes into Separate Columns

Create separate columns for Email and Phone. If a student has multiple phone numbers, create multiple rows:

StudentID	Name	Street	City	Email	Phone
101	Alice	12A	NY	alice@mail.com	12345
102	Bob	45B	LA	bob@mail.com	NULL
103	Charlie	78C	SF	charlie@mail.com	67890
103	Charlie	78C	SF	charlie@mail.com	54321

Observations:

- All attributes are now atomic (1NF requirement satisfied).
- Composite attributes (Address) are split into separate columns.
- Nested/multivalued attributes (Contacts) are split into separate atomic columns.
- Multiple phone numbers for the same student result in multiple rows.
- Duplication of student information (like Name, Street, City, Email) is acceptable in 1NF.

3 Second Normal Form (2NF)

Second Normal Form builds on First Normal Form (1NF) and uses the concepts of **functional dependencies (FDs)** and **primary keys** to eliminate certain types of redundancy.

3.1 Definitions

- **Prime attribute:** An attribute that is a member of the primary key.
- **Full functional dependency:** A functional dependency $Y \rightarrow Z$ is a full functional dependency if **removal of any attribute from Y means the dependency no longer holds**.
- **Partial dependency:** If a non-prime attribute is functionally dependent on **part of a composite primary key**, it is a partial dependency.

3.2 Detailed Example: Students Enrolled in Courses

Database Meaning: This database stores information about students, the courses they are enrolled in, and the instructor assigned to each course.

1NF Table (Unnormalized for 2NF check):

StudentID	CourseID	StudentName	CourseName	Instructor
101	C01	Alice	Math	Dr. Smith
101	C02	Alice	Physics	Dr. Brown
102	C01	Bob	Math	Dr. Smith
103	C02	Charlie	Physics	Dr. Brown
103	C03	Charlie	Chemistry	Dr. Green

Step 1: Identify the Primary Key

- Composite Primary Key: $\{StudentID, CourseID\}$
- Prime Attributes: StudentID, CourseID
- Non-prime Attributes: StudentName, CourseName, Instructor

Step 2: Check Functional Dependencies (FDs)

1. **StudentID $\rightarrow$ StudentName** - Each student has exactly one name $\rightarrow$ 1:1 relationship - Partial dependency? Yes, depends only on part of the composite key $\rightarrow$ violates 2NF
2. **CourseID $\rightarrow$ CourseName, Instructor** - Each course has a unique name and instructor $\rightarrow$ 1:1 relationship - Partial dependency? Yes, depends only on part of the composite key $\rightarrow$ violates 2NF
3. **$\{StudentID, CourseID\} \rightarrow$ none other** - No other attributes depend fully on the whole composite key

Step 3: Convert to 2NF

1. **Student Table** (StudentID $\rightarrow$ StudentName)

StudentID	StudentName
101	Alice
102	Bob
103	Charlie

2. **Course Table** (CourseID $\rightarrow$ CourseName, Instructor)

CourseID	CourseName	Instructor
C01	Math	Dr. Smith
C02	Physics	Dr. Brown
C03	Chemistry	Dr. Green

3. Enrollment Table (Composite Key = {StudentID, CourseID})

StudentID	CourseID
101	C01
101	C02
102	C01
103	C02
103	C03

3.3 Step 4: Verify 2NF Compliance

- **Student Table:** StudentName fully depends on StudentID $\rightarrow$ 2NF satisfied
- **Course Table:** CourseName and Instructor fully depend on CourseID $\rightarrow$ 2NF satisfied
- **Enrollment Table:** Only prime attributes in composite key, no non-prime attributes $\rightarrow$ 2NF satisfied

3.4 Example 2: Employee-Project Database

1NF Table:

EmpID	ProjectID	EmpName	ProjectName	DeptName
E01	P01	John	Alpha	HR
E01	P02	John	Beta	HR
E02	P01	Mary	Alpha	Finance
E03	P03	Steve	Gamma	IT
E03	P02	Steve	Beta	IT

Primary Key: {EmpID, ProjectID}

Checking FDs:

- EmpID $\rightarrow$ EmpName, DeptName $\rightarrow$ Partial dependency
- ProjectID $\rightarrow$ ProjectName $\rightarrow$ Partial dependency

Decomposition to 2NF:

- Employee Table (EmpID $\rightarrow$ EmpName, DeptName)
- Project Table (ProjectID $\rightarrow$ ProjectName)
- Assignment Table (EmpID, ProjectID)

—

3.5 Example 3: Orders Database

— **1NF Table:**

OrderID	ProductID	CustomerName	ProductName	Category
O01	PR01	Alice	Laptop	Electronics
O01	PR02	Alice	Mouse	Electronics
O02	PR01	Bob	Laptop	Electronics
O03	PR03	Charlie	Chair	Furniture

Primary Key: $\{OrderID, ProductID\}$

Checking FDs:

- $OrderID \rightarrow CustomerName \rightarrow$ Partial dependency
- $ProductID \rightarrow ProductName, Category \rightarrow$ Partial dependency

Decomposition to 2NF:

- Customer Table ($OrderID \rightarrow CustomerName$)
- Product Table ($ProductID \rightarrow ProductName, Category$)
- Order Table ($OrderID, ProductID$)

—

3.6 Observations

- Checked every non-prime attribute against all prime attributes for full functional dependency.
- Partial dependencies in the original table caused redundancy (e.g., repeating Student-Name and CourseName).
- Decomposition into separate tables eliminated partial dependencies and achieved 2NF.

4 Third Normal Form (3NF)

4.1 Definition

A relation schema R is in **Third Normal Form (3NF)** if:

1. R is already in **Second Normal Form (2NF)**.
2. No **non-prime attribute** A in R is **transitively dependent** on the primary key.

Transitive Functional Dependency:

A functional dependency $X \rightarrow Z$ is called a **transitive dependency** if there exists a set of attributes Y such that:

$$X \rightarrow Y \text{ and } Y \rightarrow Z$$

where X is a primary key and Y is not a candidate key.

Note: If Y is a candidate key, transitive dependency does not violate 3NF.

4.2 Examples

Example 1: Employee-Department Database

Consider the relation:

$$EMP(SSN, ENAME, SALARY, DNUMBER, DMGRSSN)$$

Functional dependencies:

- $SSN \rightarrow ENAME, SALARY, DNUMBER$
- $DNUMBER \rightarrow DMGRSSN$

Step 1: Identify Prime and Non-prime Attributes

- Primary Key = SSN
- Prime Attribute = SSN
- Non-prime Attributes = ENAME, SALARY, DNUMBER, DMGRSSN

Step 2: Check for Transitive Dependencies

- $SSN \rightarrow DNUMBER$ and $DNUMBER \rightarrow DMGRSSN$ - This implies: $SSN \rightarrow DMGRSSN$ (transitive dependency) - DNUMBER is not a candidate key, so this violates 3NF.

- $SSN \rightarrow ENAME$ - No intermediate attribute $\rightarrow$ non-transitive $\rightarrow$ satisfies 3NF

Step 3: Decomposition into 3NF

1. EMP Table: SSN, ENAME, SALARY, DNUMBER

SSN	ENAME	SALARY	DNUMBER
101	John	5000	D01
102	Mary	6000	D02
103	Steve	5500	D01

2. DEPARTMENT Table: DNUMBER, DMGRSSN

DNUMBER	DMGRSSN
D01	201
D02	202

Now both tables are in 3NF:

- No non-prime attribute is transitively dependent on the primary key. - All partial dependencies were already removed in 2NF.

Example 2: Transitive vs Non-Transitive FDs

- $SSN \rightarrow DNUMBER$ and $DNUMBER \rightarrow DMGRSSN \rightarrow$ **transitive FD**
- $SSN \rightarrow ENAME \rightarrow$ **non-transitive FD** (no intermediate attribute exists)

Observation: - Any table in 2NF may still violate 3NF due to transitive dependencies. - Decomposition removes transitive dependencies while preserving all functional dependencies.

4.3 Summary of 3NF Rules

- Must be in 2NF.
- No non-prime attribute depends on another non-prime attribute via transitive dependency.
- Decompose relations to eliminate transitive dependencies while preserving data integrity.

Table 7: Summary of Normal Forms Based on Primary Keys and Corresponding Normalization

Normal Form	Test	Remedy (Normalization)
First (1NF)	Relation should have no multivalued attributes or nested relations.	Form new relations for each multivalued attribute or nested relation.
Second (2NF)	For relations where the primary key contains multiple attributes, no non-key attribute should be functionally dependent on a part of the primary key.	Decompose and set up a new relation for each partial key with its dependent attribute(s). Keep a relation with the original primary key and any attributes fully functionally dependent on it.
Third (3NF)	Relation should not have a non-key attribute functionally determined by another non-key attribute (i.e., no transitive dependency of a non-key attribute on the primary key).	Decompose and set up a relation that includes the non-key attribute(s) that functionally determine other non-key attribute(s).

5 Achieving BCNF by Decomposition

5.1 Superkey and Non-Trivial Functional Dependency

Superkey:

A superkey is a set of one or more attributes of a relation that uniquely identifies a tuple (row) in that relation.

Example:

Consider the relation:

STUDENT(*StudentID*, *Name*, *Email*)

- Candidate key: *StudentID* (minimal set that uniquely identifies a student)
- Superkeys:
 - {*StudentID*}
 - {*StudentID*, *Name*}
 - {*StudentID*, *Email*}
 - {*StudentID*, *Name*, *Email*}
- All superkeys uniquely identify a tuple, but only candidate keys are minimal.

—

Non-Trivial Functional Dependency:

A functional dependency $X \rightarrow Y$ is called non-trivial if Y is not a subset of X .

Example:

Using the *STUDENT* relation:

- $StudentID \rightarrow Name \rightarrow$ Non-trivial (Name is not part of StudentID)
- $StudentID \rightarrow \{StudentID, Name\} \rightarrow$ Trivial (StudentID is part of the right-hand side)

Intuition: Non-trivial FDs represent actual information about relationships between attributes, whereas trivial FDs are always true by definition and do not provide meaningful constraints.

—

5.2 Background: BCNF

Boyce-Codd Normal Form (BCNF) is an advanced form of normalization that builds on 3NF.

A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$:

- X is a superkey of the relation.

Intuition for Beginners:

- A determinant is the attribute(s) on the left-hand side of a functional dependency. - BCNF ensures that no attribute determines another attribute unless it is a superkey. - This eliminates redundancy and avoids anomalies that can occur even in 3NF.

5.3 Example: TEACH Relation

Consider the relation:

$TEACH(student, course, instructor)$

with the following functional dependencies:

- fd1: $\{student, course\} \rightarrow instructor$
- fd2: $instructor \rightarrow course$

Candidate Key: $\{student, course\}$

- The relation satisfies 3NF (no transitive dependencies among non-prime attributes). - However, it is not in BCNF because fd2 violates BCNF: - Determinant instructor is not a superkey.

This can cause redundancy. For example, the same instructor may appear multiple times for different students, repeating the course value unnecessarily.

5.4 Lossless Join Property (Non-Additivity)

When decomposing a relation to achieve BCNF, it is essential to maintain the lossless join property.

Definition:

A decomposition of a relation R into R_1 and R_2 is lossless (non-additive) if:

$$R_1 \bowtie R_2 = R$$

- That is, if we join the decomposed relations on their common attributes, we should get exactly the original relation, without generating extra (spurious) tuples.

Why it matters:

- If a decomposition is not lossless, joining the smaller tables may produce wrong or extra tuples, which corrupts data integrity. - BCNF decomposition cannot sacrifice this property; otherwise, the database will become inconsistent.

Intuition for Beginners:

- Think of a relation as a puzzle. - Decomposition splits it into smaller pieces. - Lossless join ensures that putting the pieces back together gives you the original puzzle exactly, with nothing extra.

5.5 BCNF Decomposition for TEACH

Possible decompositions:

1. D_1 : $\{student, instructor\}$ and $\{student, course\}$
2. D_2 : $\{course, instructor\}$ and $\{course, student\}$
3. D_3 : $\{instructor, course\}$ and $\{instructor, student\}$

Primary Keys for each decomposition:

- **D1:**

- $R_1(student, instructor)$: PK = $\{student, instructor\}$
- $R_2(student, course)$: PK = $\{student, course\}$

- **D2:**

- $R_1(course, instructor)$: PK = $\{instructor\}$
- $R_2(course, student)$: PK = $\{course, student\}$

- **D3:**

- $R_1(instructor, course)$: PK = $\{instructor\}$
- $R_2(instructor, student)$: PK = $\{instructor, student\}$

Observations:

- All three decompositions may lose fd1 (dependency preservation may be sacrificed).
- Only D3 preserves the lossless join property, avoiding spurious tuples when joining back.

—

5.6 Key Points

- BCNF requires every determinant to be a superkey.
- A superkey is any set of attributes that uniquely identifies a tuple.
- Non-trivial FDs are the ones that matter for BCNF checks.
- Decomposition to BCNF may require sacrificing functional dependency preservation, but never lossless join.
- Ensuring lossless join prevents spurious tuples and preserves original data integrity.
- Decomposition should satisfy:
 1. BCNF property (all determinants are superkeys)
 2. Lossless join (non-additivity)